

1. (20 points) (Research/Essay-like question)

- a. Give a definition of Distributed Computing System. (Hint: there is a good one in Tannenbaum's book).**

A distributed computing system means a collection of independent computers that appears to its users as a single coherent system and can be used for difficult problems which cannot be resolved by ordinary machines. In other words, a distributed computing system not only allows an user to run multiple jobs concurrently utilizing multiple distributed resources, but also provides a desired level of abstraction to the user so that the user think that he/she works on a single machine. As a result, the user can finish the job in more efficient and faster way than when he/she uses sequential algorithms on a single processor machine.

According to Tannenbaum's book, there are two major subgroups of distributed computing systems; cluster computing and grid computing. Cluster computing consists of a collection of similar machines connected by switch, hub, or router, while grid computing consists of a collection of different machines and different software connected by wide area network or Internet.

Reference:

1. Tanenbaum and M. Van Steen, "Distributed Systems: Principles and Paradigms," Second Edition, Chapter 1.
2. A. Bertolli, "A Comparison of Distributed Systems: ChorusOS and Amoeba"

- b. Discuss the general structure/architecture of such a system and discuss its different components (hardware, interconnection network, system's software, etc.).**

In class, we learned four different types of distributed architectures such as SISD (Single Instruction, Single Data), SIMD (Single Instruction, Multiple Data), MISD (Multiple Instruction, Single Data), and MIMD (Multiple Instruction, Multiple Data).

Among them, MIMD would be the most general DCS architecture since it literally supports the definition of DCS as above. In MIMD architecture, multiple machines (or processors) will be executing different instructions concurrently. MIMD architecture consists of processor, memory, interconnection network (processor-to-processor and processor-to-memory), system software which is middleware, and programming model and tools.

A. Processor:

It runs a given instruction. It can be either physical CPU or virtual processor generated by virtual machine. In MIMD, multiple processors are connected to each other and to memory modules through interconnection network.

B. Memory:

It is used to store variables that are necessary for processors to execute instructions. In MIMD, there are some local memory and a modular shared memory. Local memory can be shared by all the processors so that there is only one memory address space for all processors, or distributed so that different processors have their own memory address space. Although there are many processors in the distributed system, according to the paper by Bart Sinclair, we might not be able to see a great improvement if shared memory modules are not sufficient.

C. Interconnection network:

It connects processors to processors or processors to memory modules. When developing DCS, it is easy to forget about interconnection network speed (latency and bandwidth). In fact, they are significant factors in DCS since they differentiate between different DCS architectures. In class, we learned four types of interconnection network; cross-bar/multistage, switch/hub/router, local area network (Ethernet), and wide area network (WAN)/Internet. cross-bar is composed of several switches and used for tightly coupled DCS such as SIMD or massively parallel processors which is a specialized architecture with large number of processors. Switch/hub/router is used for cluster computing while Ethernet is used for workstation farm. For more loosely coupled DCS, such as grid computing, WAN/Internet is used.

D. System software (or OS):

In DCS, system software acts as a middleware hiding physical machines from the user's point of view and allowing the user to focus on programming within the space of his/her problem. It handles spatial scheduling (where to place the job), temporal scheduling (when to execute the job), load balancing (redistributing the job), and memory and I/O problems.

E. Programming model and tools:

Programming model depends on the programming languages. For example, Java supports stack-oriented programming model with Java Virtual Machine. Some programming languages allow us to write data parallel program. The combination of the data parallel programming model and a MIMD machine model is called a Single Program Multiple Data (SPMD) execution model. In this model, all processors execute the same program, but at different stages of the program at a any given time. In addition to programming model, there are some tools that help a developer write parallel program easier, such as Parallel Virtual Machine (PVM) that provides programming environment that emulates a traditional parallel system, Remote Procedure Call (RPC) that provides procedures or functions which can be accessed remotely, and Common Object Request Broker Architecture (CORBA) that provides C++ objects to other nodes.

Reference:

1. A. Tanenbaum and M. Van Steen, "Distributed Systems: Principles and Paradigms," Second Edition, Chapter 1.
2. Bart Sinclair, "Analysis of Shared memory multiprocessors"
3. A. Bertolli, "A Comparison of Distributed Systems: ChorusOS and Amoeba"

c. Explain the differences between tightly and loosely coupled Distributed Computer Systems. Can you relate the distinctions to the OSI Layered Model of Communication?

Tightly coupled DCS means that machines are geographically close to each other connected by interconnection network such as cross-bar, while loosely couple DCS means that machines are placed throughout a wide range of area connected by WAN/Internet.

In OSI Layered Model of Communication perspective, tightly coupled DCS may not need to have a network layer. The network layer, placed in the third from the bottom of OSI model, handles packet routing and forwarding based on IP address. This is the layer where IP comes into play. However, since computers in tightly coupled DCS are connected by switches, not by wide area network or Internet, IP address is not necessary to send data between machines.

d. Discuss the application domain of Distributed Computer Systems.

DCS can be used in kinds of problems that involve large computations which may not be solved with ordinary machines (i.e. single core or sequential algorithm). Those problems include particle physics, bioinformatics, office resource sharing, visualization & graphics rendering, enterprise management, and embedded systems (like cell phone).

Reference:

1. A. Bertolli, "A Comparison of Distributed Systems: ChorusOS and Amoeba"

2. (20 points) (Algorithm Mapping)

Given the Fast Fourier Transformation (FFT) of a linear vector array, the 2D FFT can be computed using the 1D FFT by FFTing rows and then FFTing columns of the 2D array.

a. Propose a method to compute the 3D FFT based on 1D and/or 2D FFTs.

In the easiest way, we can use 2D FFT that we learned in class and transpose the result for another 1D FFT calculation along remaining dimension. This is called 1D decomposition. For example,

1. 1D FFT the y dimension
2. local transpose within a processor
3. 1D FFT the z dimension
4. global transpose within the whole system
5. 1D FFT the x dimension

Assuming that the data is in 3D cubic structure, we may separate them into several chunks along one axis. For example, we may cut 3D cube along x-axis so that there are several yz-plane data chunks. Each data chunk will be assigned to available processor. Then, each processor will calculate 1D FFT along y-axis, transpose the chunk locally within the processor, and calculate 1D FFT along z-axis. There is no need for each processor to communicate with each other since they will only handle their own data chunk at this point. After that, all the processors in the system need to communicate with each other to transpose the whole 3D cubic structure for 1D FFT along x-axis. This is called global transpose and will be expensive since it involves each processor communicating with all the other processors in the system. Finally, each processor will finish 1D FFT along x-axis to achieve 3D FFT calculation.

Reference:

1. Roland Schulz, "3D FFT with 2D decomposition"

b. Suppose the 3D FFT is to be programmed on a cross-bar interconnected cluster. Suggest an implementation of this 3D FFT and discuss its performance. (Identify possible bottlenecks and propose solutions).

However, there is one disadvantage in this method; the number of processors that the system can utilize will be limited by the length of the largest axis of 3D cube. For example, if the largest axis is x-and we cut 3D data cube along x-axis, then the number of yz-planes will be the maximum number of processors that the system can utilize concurrently. In order words, even though we have a lot more processors than the length of the largest axis, we may not be able to fully utilize them. To increase processor utilization, we may use 2D decomposition instead of 1D decomposition above. Implementation will look as below.

1. 1D FFT the y dimension
2. global transpose within a subgroup (complexity of transfer: $O(k)$)

For all i, do parallel:

For all $j = 1, k - 1$, do:

P_i sends Block $B_{(j+i) \bmod k}$ to Processor $P_{(j+i) \bmod k}$ within subgroup

endfor

endfor

3. 1D FFT the z dimension
4. global transpose within the whole system (complexity of transfer: $O(k)$)

```

For all i, do parallel:
    For all j = 1, k - 1, do:
        Pi sends Block Bi(j+i)mod k to Processor P(j+i)mod k within the whole system
    endfor
endfor

```

5. 1D FFT the x dimension

Using the same example above, we cut 3D cube along x-axis which will be resulted in several yz-plane data chunks. In 2D decomposition, we cut those yz-plane one more time so that smaller several yz-plane data sets of one big yz-plane data chunk will be assigned to available processors. In other words, if we used one processor per yz-plane in 1D decomposition method above, in 2D decomposition we would be able to utilize multiple processors per yz-plane. Let's call those multiple processors a subgroup. After each subgroup completes 1D FFT calculation along y-axis, then each processor in the subgroup should communicate each other to transpose the yz-plane for 1D FFT calculation along z-axis. After 1D FFT along z-axis is completed, a transpose among subgroups is required for 1D FFT along x-axis. This is similar to the global transpose in 1D decomposition as all the processors in the system should communicate with each other.

In the implementation above, the most expensive bottleneck will be generated by the communication between processors, which is a global transpose. Since there are two global transposes, 2D decomposition implementation would be slower than 1D decomposition which contains only one global transpose. However, if we implement 2D decomposition on a cross-bar interconnection network, the cost of communication would be cheaper than 1D decomposition with other types of interconnection network since cross-bar interconnection network has low latency and high throughput. Moreover, with 2D decomposition, we can utilize more computing resources so that the overall performance will be better than other combinations of decomposition algorithm and interconnection network.

Reference:

1. Roland Schulz, "3D FFT with 2D decomposition"

3. (20 points) (Data Allocation, Data access in distributed data bases)

Consider a distributed data base where many computer resources are used to increase storage capacity.

a. What are the differences, if any, between a Networked File System (NTFS) and a Distributed Shared Memory System?

NFS enables an user to access to data on a remote file system over the network as if he/she works on a local file system. This is possible as a file system on remote machine will be mounted into a local machine through NFS so that the user will be able to write/read data to/from the remote machine without knowing that the machine is in a remote location.

Meanwhile, distributed shared memory system treats each physically separated memory as a part of one big global memory so that those distributed memories can be addressed as if there is only one address space. Thus, intercommunication became transparent to the end-user.

Reference:

1. <https://www.freebsd.org/doc/handbook/network-nfs.html>
2. <https://perso.ens-lyon.fr/laurent.lefevre/DOSMOS/DSM.html>

b. Access to records, measured in network delays, will depend on where the data is stored with respect to the requesting device. Discuss dynamic storage strategies for the on-line optimization of data-base-like operations on a geographically distributed data base. Relate your answer to topology as well as latency and bandwidth parameters of the network.

Dynamic storage strategies that in a geographically distributed database system, a block of a database will be dynamically reallocated (moved) or migrated (copied) to the database located near to the requesting point or node in DCS, in order to increase the performance (less access time to data). For such an online optimization, it is important to calculate the access probability of data dynamically since the data access patterns will be changing with time. One strategy is to count the frequencies of data access made by each node in DCS and migrate the fragment of data to the node that has the highest access counter value. Meanwhile, we have to consider a situation that there is only one specific node in DCS working at the moment so that all the distributed data will be moved into one database of that node. This would later degrade the performance of the whole DCS. To handle such a case, we can set a load threshold for each node to prevent one node from being overloaded with data. By utilizing dynamic storage strategies, we would be able to increase the performance of distributed database system over the network such as local area network as it reduces access time to data by reallocating it to the near place. The improvement will be more obvious in wide area network, because it takes more time to access data in longer distance.

Reference:

1. Brunstrom, S. T. Leutenegger, R. Simha, "Experimental evaluation of dynamic data allocation strategies in a distributed database with changing workloads"
2. T. Ulus and M. Uysal, "Heuristic Approach to Dynamic Data Allocation in Distributed Database Systems"

4. (20 points) (Research/System's resource management)

Find and choose two currently available Distributed Operating Systems.

- a. Using the scheduling models given in class, explain how each of the chosen Distributed Operating Systems effects the scheduling of concurrent computations.**

Amoeba:

Amoeba has a pool of processors that will be later automatically and dynamically assigned to the command that a user enters. One or more processors can be assigned to a single command. Moreover, there could be a lot of users entering various commands at the same time. Once processors are assigned, Amoeba will time-share each processor and use round-robin scheduling algorithm to execute several commands in a parallel way. After the command is completely processed, processors assigned for that command will be put back to the pool waiting for the next command.

Mosix:

In Mosix, a user can run parallel application while Mosix supports multi-user, timesharing execution environment.

Reference:

1. F. Douglass, J. K. Ousterhout, M. F. Kaashoek, A. S. Tanenbaum, "A Comparison of Two Distributed Systems: Amoeba and Sprite"
2. G. J. Sharp, A. S. Tanenbaum, "The Amoeba Distributed Operating System"
3. A. Barak, O. La'adan, "Performance of the MOSIX Parallel System for a Cluster of PC's"

- b. If these OSs implement Load Balancing, explain how they do it. Otherwise, suggest a method to incorporate Load Balancing in each.**

Amoeba:

Initially, Amoeba was not designed and implemented to support dynamic load balancing because the designers of the system thought that there always be more processors than the number of processes required by user commands.

As it turned out false, Weiping Zhu C.F. Steketee tried to build load balancing facilities into Amoeba. According to their paper, there are few factors needed to be considered for process migration in Amoeba; all threads of the process, the execution state of the process, the memory space of the process, the communication state of the process. When the source machine receives a migration request from the destination machine, those kinds of data should be transferred. Additionally, as the process would be suspended and not available for incoming message, there should be a way to let the sending machine to resend the same message after the migration is complete.

By conducting experiment, they suggested a centralized load balancing method for Amoeba because centralized load balancing, where one load balancer talks to all the other computers, generates less overhead than distributed load balancing, where computers in the system talk to each other. Additionally, they also found that when the size of a system exceeds a certain threshold, job initiation algorithm, where the load balancing occurs when a new job arrives, became more critical to the system performance than process migration algorithm, where the load balancing occurs when an imbalance is detected.

To incorporate load balancing in Amoeba, these factors should be considered.

Mosix:

In Mosix, dynamic load balancing will be provided with preemptive process migration. This indicates that process in overloaded node can be migrated at any time to any available node without noticing the user. Additionally, load balancing will be executed in a decentralized way. In other words, all the nodes in the system will execute the same load balancing algorithm, and based on that, process migration will occur independently within the pairs of nodes. Load balancing algorithm will be updated with the loads of nodes, the status of processes, and the number of nodes.

Reference:

1. W. Zhu, C.F. Steketee, "An Experimental Study of Load Balancing on Amoeba"
2. A. Barak, O. La'adan, "The MOSIX multicomputer operating system for high performance cluster computing"

5. (20 points)

Consider the fault tolerance problem.

- a. What is ABFT? Is there a systematic way of converting a concurrent computation into a ABFT one based on the algorithm itself? Can you suggest such a systematic (or a different one) approach?**

ABFT stands of algorithm-based fault tolerance developed by Huang and Abraham in 1984. The basic idea of ABFT is that input data will be encoded with checksum scheme and original algorithm will be redesigned to run on that encoded input data. Then, computational steps of algorithm will be distributed among available processors to reduce the number of data affected by an error in one processor. Then, the modified algorithm would generate encoded output which will be decoded to check the result by comparing the checksum values. If any of checksum does not match with what was expected, there would be a sign of a fault in the computing system. With ABFT, error detection and correction can be achieved concurrently with normal system operation.

In order to automate the conversion of a concurrent computation into the one with ABFT, we may need to automate following parts.

Firstly, we need to automate analysis of input data and data encoding with checksum scheme. Secondly, based on the encoded data, the modification of original algorithm also needs to be automated.

Thirdly, there should be an automated way to analyze and slice modified algorithm and then distribute each computation step of that algorithm properly to available processors.

Lastly, we need to automate decoding of encoded data at the end.

Reference:

1. K. Huang, J. A. Abraham, "Algorithm-Based Fault Tolerance for Matrix Operations"

- b. What types of faults can be tolerated using:**

- **leader election**
- **consensus**
- **solution(s) to the Byzantine Generals Problem**

These fault tolerance techniques can be generalized as a solution to an agreement problem, which requires a certain number of processors that agree with the result of computation. Such kind of solution can be used for tolerating faults such as response failure or arbitrary failure. Response failure means that some of processors respond with incorrect values or fall into unexpected flow of control. Arbitrary failure means that some of processors operate properly and generate a fault value pretending to be a correct one. In contrast to fail-stop faults that are caused by a node crash and can be detected by other nodes in the system, these kinds of Byzantine failures are not easy to detect since those nodes are still operating sending faulty values. By applying solutions such as leader election, consensus, or Byzantine Generals problem solution, we can have the system work appropriately even with the existence of faulty nodes.